Chế độ Tức thời (Eager) vs Chế độ Biểu đồ (Graph)

- TensorFlow 1.x yêu cầu định nghĩa đồ thị trước, rồi mở `tf.Session` để thực thi (Graph Execution). Quá trình debug vô cùng tàn bạo.
- TensorFlow 2.x đổi phương thức hoạt động mặc định thành **Eager Execution** - Chế độ thực thi tức thì.
- Mã chạy dòng nào có kết quả ngay. Gỡ lỗi bằng `print()` truyền thống.
- **Khuyết điểm:** Eager Mode Chậm hơn nhiều so với Graph (không thể tối ưu song song hóa phần cứng).

@tf.function: Phép màu của AutoGraph

- TensorFlow 2 mang đến giải pháp dung hòa bằng AutoGraph!
- Bạn viết code Python bình thường, chỉ cần đánh dấu `@tf.function`. TF sẽ tự động dịch thành đồ thị C++ ẩn bên dưới.

```
def cube(x):
    return x ** 3

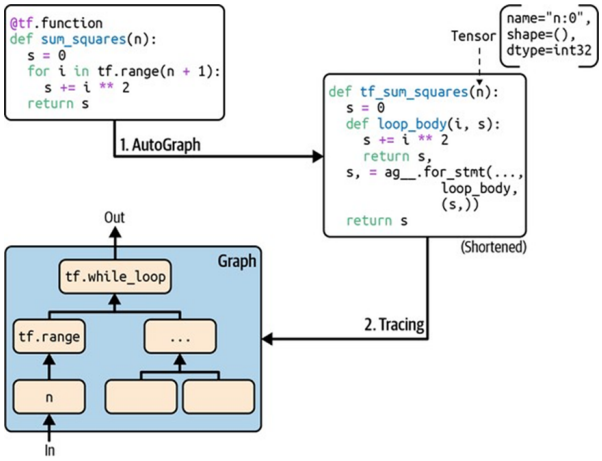
# Tinh toan - Nhanh cho Test nhung cham khi huan luyen
cube(tf.constant(2.0))

# Khai bao TF Function bang decorator
@tf.function
def tf_cube(x):
    return x ** 3

# tf_cube tro thanh mot computational graph!
tf_cube(tf.constant(2.0))
```

Cách AutoGraph tạo Biểu đồ và Tracing (1/2)

- Hình 12-4 mô tả cơ chế của AutoGraph.
- Mã Python được quét, tạo Abstract Syntax Tree.
- Sinh ra hàm Python nâng cấp gọi các toán tử điều khiển lõi của TF như `tf.cond`, `tf.while_loop`.



Hình 12-4: Cách TensorFlow tạo biểu đồ bằng AutoGraph và tracing

Cách AutoGraph tạo Biểu đồ và Tracing (2/2)

- Khi thực thi lần đầu tiên (**Tracing**), nó vạch ra lộ trình các phép toán thành **TensorFlow Graph**.
- Lần thực thi tiếp theo chạy Graph trực tiếp.
- Chú ý: TensorFlow tạo Poly-morphism (Đa hình đồ thị). Nếu gọi Input `int32`, nó tạo Graph 1. Nếu gọi `float32`, lưu thêm Graph 2.

Giới hạn và Quy tắc sống còn của TF Functions (1/2)

- **Side effects:** Hàm `print()` hoặc thao tác IO **chỉ chạy một lần duy nhất** ở giai đoạn Tracing, không nằm trong Đồ thị.

```
@tf.function
def my_func(x):
    print("Ham dang duoc trace:", x.dtype)
    return x + 1

my_func(tf.constant(1.0)) # In log, Tao Graph float32
my_func(tf.constant(2.0)) # KHONG in log, Tai su dung Graph float32
my_func(tf.constant(10))  # In log, Tao Graph int32
```

- Muốn in log động? Sử dụng `tf.print()`.

Giới hạn và Quy tắc sống còn của TF Functions (2/2)

- **Biến ngoài hàm / List Python:** Nếu dùng `list.append()`, list bị "đóng băng" ở trace. Hãy dùng `tf.TensorArray`.
- Tránh sinh số ngẫu nhiên bằng Numpy (`np.random.rand()`), hãy sử dụng `tf.random`.

```
@tf.function
def add_random_noise(x):
    # SAI LAM: Gia tri ngau nhien bi dong bang
    # noise = np.random.randn()

    # CHUAN XAC: Gia tri ngau nhien sinh ra moi khi goi Graph
    noise = tf.random.normal(shape=[])
    return x + noise
```

Vòng lặp huấn luyện tùy chỉnh

- Phương thức `model.fit()` tiện lợi, nhưng không phải kiến trúc nào cũng xài được (VD: mạng GAN huấn luyện Generator 1 bước, Discriminator 1 bước).
- Viết vòng lặp tùy chỉnh (Custom Training Loop) mất công, nhưng cho phép tùy biến không giới hạn.

tf.GradientTape(): Tính đạo hàm tự động

- TensorFlow cung cấp GradientTape đóng vai trò "băng ghi âm" lưu vết Tensor để lan truyền ngược (Backpropagation) tính đạo hàm tự động.

```
def f(w1, w2):
    return 3 * w1 ** 2 + 2 * w1 * w2

w1, w2 = tf.Variable(5.), tf.Variable(3.)

with tf.GradientTape() as tape:
    z = f(w1, w2)

# Yeu cau Tape tinh dao ham Z theo bien w1, w2
gradients = tape.gradient(z, [w1, w2])
print(gradients)
# Ket qua: dz/dw1=36, dz/dw2=10
```

GradientTape nâng cao

- Mặc định, ‘GradientTape‘ chỉ theo dõi biến ‘tf.Variable‘. Không theo dõi ‘tf.constant‘.
- Khi gọi phương thức ‘gradient()‘, bộ nhớ Tape bị xóa.
- Tính nhiều lần đạo hàm (VD: Hessian), dùng cờ ‘persistent=True‘.

```
with tf.GradientTape(persistent=True) as tape:
    z = f(w1, w2)

dz_dw1 = tape.gradient(z, w1)
dz_dw2 = tape.gradient(z, w2)  # Van OK

# Giai phong bo nho bang tay (bat buoc)
del tape
```

Viết Vòng lặp huấn luyện - Khởi tạo

- Tạo mô hình, cấu hình siêu tham số, Optimizer, Metric và Dữ liệu.

```
l2_reg = keras.regularizers.l2(0.05)
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="elu", kernel_regularizer=l2_reg),
    keras.layers.Dense(1, kernel_regularizer=l2_reg)
])

batch_size = 32
train_set = tf.data.Dataset.from_tensor_slices((X_train, y_train))
train_set = train_set.shuffle(len(X_train)).batch(batch_size)

optimizer = keras.optimizers.Nadam(learning_rate=0.01)
loss_fn = keras.losses.mean_squared_error
mean_loss = keras.metrics.Mean()
```

Viết Vòng lặp huấn luyện - Thực thi (Epochs & Steps)

- Lặp qua Epochs, trong Epochs lặp qua DataBatches.

```
epochs = 5
for epoch in range(epochs):
    for step, (X_batch, y_batch) in enumerate(train_set):

        # 1. Forward pass ghi bang Tape
        with tf.GradientTape() as tape:
            y_pred = model(X_batch, training=True)
            main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
            loss = tf.add_n([main_loss] + model.losses)

        # 2. Tinh Gradient theo model variables
        gradients = tape.gradient(loss, model.trainable_variables)

        # 3. Kich hoat phuong thuc cap nhat trong so (Optimize)
        optimizer.apply_gradients(zip(gradients, model.trainable_variables))
```

Cấu trúc hoàn chỉnh và Những lưu ý quan trọng

- Bạn phải chịu trách nhiệm đảm bảo mọi thứ chính xác:
  1. Truyền cờ `training=True` vào lệnh dự đoán của model để *BatchNormalization*/*Dropout* hoạt động đúng.
  2. Theo dõi Metric thủ công (`update_state(loss)`), in ra cuối Epoch và gọi `reset_states()`.
  3. Xử lý cập nhật biến hệ thống (Non-trainable variables, như trung bình trượt của Batch Norm).

Tối ưu hóa Vòng lặp với @tf.function

- Vòng lặp `for` Python qua DataBatches siêu chậm.
- Lời khuyên: Đóng gói quá trình xử lý 1 Bước (1 Step) thành `@tf.function` để tăng tốc biên dịch toán học C++.

```
@tf.function
def train_step(X_batch, y_batch):
    with tf.GradientTape() as tape:
        y_pred = model(X_batch, training=True)
        main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
        loss = tf.add_n([main_loss] + model.losses)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

# Trong vong lap chi goi train_step(X_batch, y_batch)
```

Tóm tắt

- **Cốt lõi TensorFlow** đại lượng Tensors cực kỳ giống NumPy, tối ưu đa phần cứng (GPU/TPU).
- Việc **tùy biến Keras (Custom Component)** cho phép người học và nghiên cứu làm mới hoàn toàn Losses, Metrics, Layers, Models.
- **AutoGraph (`@tf.function`)** giải quyết sự tranh chấp linh hoạt kịch bản (Python) với biên dịch nhanh chóng.
- **`tf.GradientTape`** đưa quyền kiểm soát hoàn toàn tới luồng huấn luyện cho các nghiên cứu mới.